# CSC 466 Project Update 1

Fixed and Adaptive Input Timeouts for a Phone-Controlled Multiplayer Game

David Mooers

## 1. Project Goal

I am building a simple Unity game where players use their phones as controllers and share one game screen. My project focuses on what happens when a controller temporarily stops sending input updates. If the host keeps applying the last input, a player could continue moving after releasing a control. However, stopping movement too quickly could interrupt someone who is still holding the control but whose updates have been delayed.

I will compare fixed and adaptive input timeouts to see how they handle these two situations. The fixed policy stops applying input after a set period without updates. The adaptive policy will adjust this period using the timing of recently received updates.

## 2. Related Work

AirConsole is an existing example of using smartphones as controllers for games displayed on a shared screen [1]. It provides a reference for the overall setup of my project, although my investigation focuses specifically on how the host handles gaps in controller updates.

The phi accrual failure detector provides a related approach to deciding when missing messages should be treated as a problem [2]. It uses previous heartbeat-arrival intervals to calculate a level of suspicion that a process has failed. My project does not need to declare a controller disconnected whenever an update is late, but the idea of using previous arrival times is relevant to deciding when its movement input should expire.

RFC 6298 describes how TCP calculates its retransmission timeout using smoothed round-trip time and an estimate of timing variation [3]. My adaptive policy will use input-arrival intervals rather than round-trip time. I am considering a similar approach of combining an estimated interval with an allowance for variation, rather than directly implementing TCP's retransmission algorithm.

## 3. Current Implementation

I have implemented a Unity prototype using Unity Transport's WebSocket network interface. The host listens on port 7777 and moves a cube using the latest movement

input received from a controller. The current controller uses WASD for local testing. Phone controls have not yet been tested in this baseline.

The controller sends input at a configurable rate. Each message contains a one-byte message type, an unsigned sequence number, and two floating-point values representing horizontal and vertical movement. It continues sending updates even when the input has not changed. This allows the host to distinguish a held input from a gap in updates.

Pressing P pauses or resumes input transmission while leaving the connection open. This gives me a simple way to test how the host behaves when updates stop arriving.

The host records when each input message arrives. If the time since the latest message exceeds the configured timeout, it stops applying that input. Movement can resume when a new message arrives. The host also displays the number of connected controllers, the age of the latest input, the timeout threshold, and whether the input is ACTIVE or TIMED OUT.

I added a menu scene so that separate Multiplayer Play Mode instances can choose to run as the host or controller. This allows me to test both locally without rebuilding the application after each change. The current implementation uses one shared player and input state; separate players for multiple controllers are not yet implemented.

## 4. Initial Test and Results

I ran an initial test with one host and one controller on the same computer using Multiplayer Play Mode. The controller was configured to send at 30 Hz, and the host used a fixed timeout of 500 ms.

During the test, I used WASD to move the cube and pressed P several times to pause and resume transmission. The host logged each input arrival and each transition into or out of the timeout state to a CSV file.

| **Measure** | **Result** |
|---|---|
| Configured send rate | 30 Hz |
| Configured fixed timeout | 500 ms |
| Input messages recorded | 436 |

| | |
|---|---|
| Input-arrival intervals measured (pauses excluded) | 428 |
| Mean input-arrival interval (pauses excluded) | 33.38 ms |
| Arrival-interval standard deviation | 4.40 ms |
| Timeout events | 7 |
| Recoveries after timeout | 7 |
| Mean input age when timeout was detected | 502.75 ms |
| Input-age range when timeout was detected | 500.10 - 505.90 ms |

The 436 received messages produced 435 arrival intervals, since the first message had no earlier message to compare against. Seven intervals included deliberate pauses in transmission, so I excluded them when calculating the mean and standard deviation.

The mean arrival interval was close to the 33.3 ms expected at a send rate of 30 Hz. This measures the time between messages arriving at the host, rather than how long each message takes to travel from the controller.

The host timed out seven times and resumed accepting input after each pause ended. On average, it detected the timeout 2.75 ms after the configured threshold. This small delay is consistent with checking the timeout once per frame.

These results provide an initial check that the fixed timeout and logging are working. Further testing is needed to compare timeout settings and determine whether an adaptive policy performs better.

## 5. Limitations and Next Steps

Testing so far has used a keyboard controller and a host running on the same computer. I have not yet tested phone-browser input, multiple independently controlled players, or controlled network delay, jitter, and loss. Pausing transmission creates an application-level gap in updates. It does not reproduce all the effects of a network interruption.

My next step is to implement an adaptive timeout based on recent input-arrival intervals and their variation, with minimum and maximum limits. I will compare it with several fixed timeout settings across different send rates and controlled network conditions.

To make the comparison repeatable, I plan to use scripted input sequences with known hold and release times. This will let me measure both unwanted movement after a release and false stops while movement is still intended. I will also record timeout events and arrival intervals to help explain the differences between the policies.

## References


[1] AirConsole, “AirConsole Guidelines for Developers.” [Online]. Available: https://documentation.airconsole.com/. Accessed: Oct. 27, 2026.

[2] N. Hayashibara, X. Défago, R. Yared, and T. Katayama, “The φ accrual failure detector,” Japan Advanced Institute of Science and Technology, Tech. Rep. IS-RR-2004-010, May 2004. [Online]. Available: https://hdl.handle.net/10119/4784.

[3] V. Paxson, M. Allman, J. Chu, and M. Sargent, “Computing TCP’s Retransmission Timer,” RFC 6298, June 2011. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6298.html.